Metro Transportation Analytics

1. Project Overview

This project analyzes Metro transaction data to understand passenger activity, fare revenue, station and line performance, travel-time patterns, and passenger/payment segments. The workflow connects **Excel → Python → SQL → Power BI** to turn transaction-level data into business insights.

2. Dataset Summary

- **Records:** 19,998
- **Distinct transaction IDs:** 19,997
- **Period:** 1 January–31 December 2025
- **Entry/exit stations:** 46 each
- **Metro lines:** Red and Blue
- **Average fare:** 36.05
- **Total fare value:** 720,903.76
- **Average journey duration:** 36.19 minutes

Note: Fare currency is not documented in the supplied files. These are descriptive results from the provided dataset and are not independently verified live Metro operating figures.

3. Data Preparation and Python Analysis

Excel was used for initial review and validation. Python/pandas was used to profile the dataset, prepare derived fields, and summarize

demand and fare patterns.

The analytical CSV includes journey details, fare and distance, passenger and ticket attributes, payment mode, date/time features, and data-quality flags.

Data-quality items to review:

- One duplicate transaction ID.
- Two non-positive fare values.
- Two ages outside a basic 0–120 range.
- Three duration-validation values marked `#REF!` or `Check`.
- Inconsistent peak-hour fields: `Peak flag`, `Peak_Category`, and `Peak_Flag` do not agree.
- The `Revenue` column has a leading space and should be checked against `Fare`.

These issues should be investigated and resolved or explicitly documented before final reporting.

4. SQL Business Analysis

The SQL script is intended to analyze:

- Total and distinct transactions, fare totals, and average fare.
- Daily and hourly demand.
- Top entry stations and Metro line performance.
- Peak versus off-peak activity.
- Passenger, ticket, and payment-mode segments.
- Station activity and route patterns.

Implementation note: the SQL file uses inconsistent table names (`metro_trans`, `metro_trips`, and `Metro_Transactions`). Standardize the table name and confirm SQL dialect and column names before running the full script.

5. Key Findings

Based on `Metro_cmpt_cleaned_data_fixed.csv`:

- **Line performance:** Red recorded 10,806 transactions and fare value of 389,665.54; Blue recorded 9,192 transactions and 331,238.22.
- **Top entry station:** HITEC City had 627 records, followed by KPHB Colony (607) and MG Bus Station (604).
- **Payment mode:** UPI was most common with 8,293 records, followed by Smart Card (5,967).
- **Passenger type:** Adult was the largest group with 13,585 records.
- **Ticket type:** Single Journey was the most common ticket type with 11,014 records.
- **Hourly activity:** hour 10 had the most records (1,815), followed by hour 21 (1,736) and hour 20 (1,733).

These are descriptive patterns in the supplied file. Station rankings are based on `Entry_Station` only, and peak-period conclusions should be withheld until peak fields are corrected.

6. Power BI Dashboard

The project README and guide describe a three-page management report:

1. **Executive Overview:** total trips, fare value, average fare/duration, monthly trend, line revenue, top stations, and filters.
2. **Station & Route Performance:** station rankings, line comparison, station-by-line matrix, and trips-versus-revenue analysis.
3. **Operations & Customer Analysis:** hourly demand, peak periods, weekday/weekend activity, passenger types, ticket types, and payment modes.

The `.pbix` file was supplied, but its internal visuals, relationships, and DAX measures were not independently verified for this report. Confirm the implemented pages and measures in Power BI Desktop.

7. Business Recommendations

- Resolve duplicate IDs, invalid fares, implausible ages, and duration validation errors.
- Create one agreed peak/off-peak rule and apply it consistently across Python, SQL, and Power BI.
- Investigate high-volume stations using entry and exit counts plus actual capacity and service data before proposing operational changes.
- Compare line-level transaction counts, average fare, distance, and duration before interpreting fare differences.
- Monitor payment-mode mix and use additional settlement or failed-payment data if available.
- Reconcile row counts, distinct IDs, date range, and fare totals at each stage of the pipeline.

8. Limitations

- The raw workbook mentioned in the README was not supplied, so the full cleaning process could not be reproduced from raw data.
- The data source and whether the dataset is synthetic or production data are not documented.
- Fare currency and revenue accounting definitions are not specified.
- Peak classifications conflict and require correction.
- SQL table names need standardization.
- The Power BI report contents were not independently inspected.
- `Line_Performance.xls` appears incomplete and should be re-exported if needed.

9. Conclusion

This project demonstrates an end-to-end analytics workflow using Excel, Python, SQL, and Power BI. The supplied dataset supports descriptive analysis of transaction volume, fare value, station activity, line performance, hourly demand, and customer/payment segments. Before

presenting final management conclusions, fix the identified data-quality issues and validate that key metrics reconcile across all four tools.